

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 6: Divide and Conquer I: Mergesort and Recurrences

Foreign Trade University

Contents

1	Introduction: The Divide-and-Conquer Paradigm	1
1.1	Why divide and conquer wins	2
2	Mergesort	2
2.1	The algorithm	2
2.2	Correctness	2
2.3	Running time: $O(n \log n)$	3
3	Counting Inversions	4
3.1	The problem	4
3.2	Counting inversions during a merge	4
4	Solving Recurrences	5
4.1	Unrolling	5
4.2	Recursion trees	5
4.3	The Master Theorem	5
5	Binary Search	6
6	Maximum-Subarray Sum via Divide and Conquer	7
7	Fast Exponentiation	7
8	Quickselect: Expected Linear-Time Selection	8
9	Two More Divide-and-Conquer Patterns	9
9.1	Majority element	9
9.2	Peak finding in 1-D	9
10	Karatsuba vs. Grade-School Multiplication (a recurrence comparison)	9
11	Summary and Looking Ahead	9

1 Introduction: The Divide-and-Conquer Paradigm

This week begins our study of **divide-and-conquer algorithms**: algorithms that solve a problem by *dividing* it into smaller subproblems of the same kind, *conquering* (solving) those subproblems recursively, and then *combining* their solutions into a solution for the original problem.

The pattern is so common that it has a fixed three-step shape:

- **Divide.** Split the input of size n into a subproblems, each of size roughly n/b (often $a = b = 2$: two halves).
- **Conquer.** Solve each subproblem recursively. The recursion bottoms out at a *base case* – inputs small enough to solve directly (e.g. a single element, which is trivially sorted).
- **Combine.** Merge the subproblem solutions into a solution for the whole, doing some additional work (the *combine step*).

The running time of such an algorithm satisfies a **recurrence** of the form

$$T(n) = aT(n/b) + (\text{cost of dividing and combining}),$$

and most of the analytical work this week is about *solving* such recurrences – by unrolling them, drawing recursion trees, or applying the **Master Theorem**.

1.1 Why divide and conquer wins

A recurring theme is that divide-and-conquer often beats the obvious quadratic approach. Mergesort sorts in $O(n \log n)$ rather than the $O(n^2)$ of insertion or selection sort; counting inversions drops from $O(n^2)$ to $O(n \log n)$; fast exponentiation computes x^n with $O(\log n)$ multiplications instead of $n - 1$; and (next week) Karatsuba multiplies two n -digit integers in $O(n^{\log_2 3}) \approx O(n^{1.585})$ rather than $O(n^2)$. In every case the speed-up comes from the same source: the recursion replaces a single large amount of work with a logarithmic number of *levels*, each doing only a modest amount of combine work.

This course follows Kleinberg & Tardos, Chapter 5. Today we will see mergesort, counting inversions, the toolkit for solving recurrences (including the Master Theorem), binary search, the maximum-subarray problem, fast exponentiation, and quickselect.

2 Mergesort

2.1 The algorithm

Mergesort is the canonical divide-and-conquer sorting algorithm. To sort a list of n elements: split it into two halves, recursively sort each half, then *merge* the two sorted halves into one sorted list. The base case is a list of length 0 or 1, which is already sorted.

The heart of the algorithm is the MERGE subroutine, which takes two *already-sorted* lists and interleaves them into one sorted list in linear time, by repeatedly comparing the two front elements and emitting the smaller one.

Problem 1 asks you to implement MERGE, and Problem 2 asks you to implement MERGESORT (with a comparison counter, see Section 4).

2.2 Correctness

Lemma 2.1. *If L and R are each sorted in non-decreasing order, then $\text{MERGE}(L, R)$ returns a sorted list containing exactly the multiset union of the elements of L and R .*

Algorithm 1 Merge two sorted lists

```
1: function MERGE( $L, R$ ) ▷  $L, R$  are each sorted ascending
2:    $M \leftarrow$  empty list;  $i \leftarrow 0$ ;  $j \leftarrow 0$ 
3:   while  $i < |L|$  and  $j < |R|$  do
4:     if  $L[i] \leq R[j]$  then
5:       append  $L[i]$  to  $M$ ;  $i \leftarrow i + 1$ 
6:     else
7:       append  $R[j]$  to  $M$ ;  $j \leftarrow j + 1$ 
8:     end if
9:   end while
10:  append the remaining elements of  $L$  (from  $i$ ) and of  $R$  (from  $j$ ) to  $M$ 
11:  return  $M$ 
12: end function
```

Algorithm 2 Mergesort

```
1: function MERGESORT( $A$ )
2:   if  $|A| \leq 1$  then
3:     return  $A$  ▷ base case: already sorted
4:   end if
5:    $m \leftarrow \lfloor |A|/2 \rfloor$ 
6:    $L \leftarrow$  MERGESORT( $A[0 : m]$ )
7:    $R \leftarrow$  MERGESORT( $A[m : ]$ )
8:   return MERGE( $L, R$ )
9: end function
```

Proof. Every element of L and R is appended to M exactly once (each index advances from 0 past the end of its list), so M contains the correct multiset. For sortedness, consider the loop invariant: *at the start of each iteration, M is sorted, and every element already in M is $\leq$ every element still remaining in $L[i:]$ and $R[j:]$.* Initially M is empty and the invariant holds vacuously. Each iteration appends $\min(L[i], R[j])$ – the smallest remaining element overall (since L and R are sorted, their smallest remaining elements are $L[i]$ and $R[j]$). This element is $\geq$ everything already in M (by the invariant) and $\leq$ everything that remains, so the invariant is preserved. When the loop ends, the leftover tail of one list (all $\geq$ everything in M , and internally sorted) is appended, leaving M sorted. $\square$

Theorem 2.2. MERGESORT(A) returns a sorted permutation of A .

Proof. By strong induction on $n = |A|$. *Base case:* if $n \leq 1$, A is already sorted and is returned unchanged. *Inductive step:* for $n \geq 2$, the two halves each have size $< n$, so by the inductive hypothesis L and R are returned sorted. By Lemma 2.1, MERGE(L, R) is a sorted permutation of $L \cup R = A$. $\square$

2.3 Running time: $O(n \log n)$

Theorem 2.3. MERGESORT runs in $O(n \log n)$ time and performs $O(n \log n)$ comparisons.

Proof. MERGE(L, R) does at most $|L| + |R| - 1$ comparisons and $O(|L| + |R|)$ total work. So the running time satisfies

$$T(n) \leq 2T(n/2) + cn$$

for a constant c , with $T(1) = O(1)$. We solve this by the recursion-tree method (Section 4). The tree has $\log_2 n$ levels; at level k there are 2^k subproblems each of size $n/2^k$, contributing $2^k \cdot c(n/2^k) = cn$ work. Summing over the $\log_2 n + 1$ levels gives $cn(\log_2 n + 1) = O(n \log n)$. By the Master Theorem (Theorem 4.1, Case 2 with $a = b = 2$, $d = 1$), $T(n) = \Theta(n \log n)$. $\square$

Mergesort's $O(n \log n)$ is a dramatic improvement over the $\Theta(n^2)$ of insertion and selection sort, and it matches the comparison-sorting lower bound of $\Omega(n \log n)$. Problem 5 asks you to instrument mergesort and confirm empirically that the *number of MERGE calls* and the total comparison count match the recurrence.

3 Counting Inversions

3.1 The problem

Definition 3.1 (Inversion). *Given an array $A[0:n]$, an inversion is a pair of indices (i, j) with $i < j$ but $A[i] > A[j]$ – a pair that is “out of order.”*

The number of inversions measures how far an array is from sorted (a sorted array has 0 inversions; a reverse-sorted array of length n has the maximum $\binom{n}{2}$). It is the basis of the *Kendall tau* rank-correlation statistic: given two people's rankings of n movies, the number of pairs they disagree on is exactly the number of inversions between the two orderings (Kleinberg & Tardos, Section 5.3).

The brute-force algorithm checks all $\binom{n}{2}$ pairs in $\Theta(n^2)$. We can do better.

3.2 Counting inversions during a merge

The key insight: *piggyback on mergesort*. Sort the array with mergesort, but during each MERGE, count the inversions between the left and right halves. When merging sorted halves L and R , whenever we emit an element $R[j]$ from the right *before* some elements still remaining in L , every one of those remaining $|L| - i$ elements forms an inversion with $R[j]$ (each is to the *left* of $R[j]$ in the original array, yet larger).

Algorithm 3 Merge-and-count

```

1: function MERGEANDCOUNT( $L, R$ )  $\triangleright L, R$  sorted
2:    $M \leftarrow \text{empty}; i \leftarrow 0; j \leftarrow 0; \text{inv} \leftarrow 0$ 
3:   while  $i < |L|$  and  $j < |R|$  do
4:     if  $L[i] \leq R[j]$  then
5:       append  $L[i]$  to  $M$ ;  $i \leftarrow i + 1$ 
6:     else
7:       append  $R[j]$  to  $M$ ;  $j \leftarrow j + 1$ 
8:        $\text{inv} \leftarrow \text{inv} + (|L| - i)$   $\triangleright R[j]$  jumps ahead of the rest of  $L$ 
9:     end if
10:  end while
11:  append leftovers; return ( $M, \text{inv}$ )
12: end function

```

The total inversion count is the sum of inversions counted inside the left half, inside the right half, and across the two during the merge:

$$\text{inv}(A) = \text{inv}(L) + \text{inv}(R) + \text{cross}(L, R).$$

Since this adds only $O(1)$ bookkeeping per merge step, the recurrence is identical to mergesort's, $T(n) = 2T(n/2) + O(n)$, giving $\Theta(n \log n)$. Problem 3 asks you to implement this; verify it against the $O(n^2)$ brute force from `starter_code.py`.

4 Solving Recurrences

Divide-and-conquer running times are described by recurrences. We need systematic ways to turn a recurrence into a closed-form asymptotic bound. We cover three: *unrolling*, *recursion trees*, and the *Master Theorem*.

4.1 Unrolling

To *unroll* a recurrence, repeatedly substitute it into itself and look for a pattern. For $T(n) = 2T(n/2) + cn$:

$$\begin{aligned} T(n) &= 2T(n/2) + cn \\ &= 2(2T(n/4) + cn/2) + cn = 4T(n/4) + 2cn \\ &= 8T(n/8) + 3cn = \dots = 2^k T(n/2^k) + kcn. \end{aligned}$$

The recursion bottoms out when $n/2^k = 1$, i.e. $k = \log_2 n$. At that point $2^k T(1) = n \cdot O(1) = O(n)$, and the accumulated combine cost is $kcn = cn \log_2 n$. Hence $T(n) = \Theta(n \log n)$.

4.2 Recursion trees

A **recursion tree** makes the same calculation visual. The root is the top-level call on size n ; it has a children of size n/b , each of which has a children of size n/b^2 , and so on. Annotate each node with the *non-recursive* (combine) work it does, then sum the work *level by level*.

For $T(n) = aT(n/b) + cn^d$:

- Level k has a^k nodes, each of size n/b^k , each doing $c(n/b^k)^d$ work.
- Total work at level k is $a^k \cdot c(n/b^k)^d = cn^d \cdot (a/b^d)^k$.
- There are $\log_b n + 1$ levels.

The total is a *geometric series* in the ratio $r = a/b^d$. The behaviour depends entirely on whether $r < 1$ (top-heavy: root dominates), $r = 1$ (balanced: every level costs the same), or $r > 1$ (bottom-heavy: leaves dominate) – which is exactly the trichotomy of the Master Theorem.

4.3 The Master Theorem

Theorem 4.1 (Master Theorem). *Let $a \geq 1$ and $b > 1$ be constants, and let $T(n) = aT(n/b) + \Theta(n^d)$ for $d \geq 0$, with a constant base case. Comparing d against $\log_b a$ gives three cases:*

- Case 1.** *If $d < \log_b a$ (equivalently $a > b^d$): $T(n) = \Theta(n^{\log_b a})$. (Leaves dominate; the work grows geometrically toward the bottom of the tree.)*
- Case 2.** *If $d = \log_b a$ (equivalently $a = b^d$): $T(n) = \Theta(n^d \log n) = \Theta(n^{\log_b a} \log n)$. (Every level does equal work; multiply by the $\log_b n$ levels.)*
- Case 3.** *If $d > \log_b a$ (equivalently $a < b^d$): $T(n) = \Theta(n^d)$. (The root's combine step dominates; the top level does the most work.)*

Proof sketch. By the recursion-tree analysis above, the total work is $\Theta(n^d \sum_{k=0}^{\log_b n} r^k)$ with $r = a/b^d$. If $r < 1$ the sum is $\Theta(1)$, giving $\Theta(n^d)$ – but here $r < 1 \iff a < b^d \iff d > \log_b a$, Case 3. If $r = 1$ the sum is $\Theta(\log_b n)$, giving $\Theta(n^d \log n)$, Case 2. If $r > 1$ the sum is $\Theta(r^{\log_b n}) = \Theta(a^{\log_b n} / n^d) = \Theta(n^{\log_b a} / n^d)$, giving $\Theta(n^{\log_b a})$, Case 1. (The identity $a^{\log_b n} = n^{\log_b a}$ is the key algebraic step.) $\square$

Example 4.2 (Worked applications).

- **Mergesort:** $T(n) = 2T(n/2) + \Theta(n)$. Here $a = 2$, $b = 2$, $d = 1$; $\log_b a = 1 = d$, so Case 2: $\Theta(n \log n)$.
- **Binary search:** $T(n) = T(n/2) + \Theta(1)$. Here $a = 1$, $b = 2$, $d = 0$; $\log_b a = 0 = d$, Case 2: $\Theta(\log n)$.
- **Karatsuba:** $T(n) = 3T(n/2) + \Theta(n)$. Here $a = 3$, $b = 2$, $d = 1$; $\log_2 3 \approx 1.585 > 1 = d$, Case 1: $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$.
- **Grade-school multiply (D&C):** $T(n) = 4T(n/2) + \Theta(n)$. Here $\log_2 4 = 2 > 1$, Case 1: $\Theta(n^2)$ – no improvement, which is exactly why Karatsuba’s drop from 4 to 3 subproblems matters.
- **A top-heavy example:** $T(n) = 2T(n/2) + \Theta(n^2)$. Here $\log_2 2 = 1 < 2 = d$, Case 3: $\Theta(n^2)$.

Problem 4 asks you to implement a *Master Theorem classifier* that, given a , b , and d , returns the correct Θ -class string. (The standard form of the theorem does not cover every recurrence – e.g. when a or b is not constant, or the combine term is not $\Theta(n^d)$ – but it handles the overwhelming majority of divide-and-conquer recurrences you will meet.)

5 Binary Search

Binary search finds a target in a *sorted* array by repeatedly halving the search interval: compare the target to the middle element; if equal, done; if the target is smaller, recurse on the left half; otherwise recurse on the right half. This is divide-and-conquer with $a = 1$ (only one subproblem is explored) and a $\Theta(1)$ combine.

Algorithm 4 Binary search (recursive)

```

1: function BINARYSEARCH( $A, x, lo, hi$ )
2:   if  $lo > hi$  then
3:     return  $-1$  ▷ not found
4:   end if
5:    $m \leftarrow \lfloor (lo + hi) / 2 \rfloor$ 
6:   if  $A[m] = x$  then
7:     return  $m$ 
8:   else if  $A[m] > x$  then
9:     return BINARYSEARCH( $A, x, lo, m - 1$ )
10:  else
11:    return BINARYSEARCH( $A, x, m + 1, hi$ )
12:  end if
13: end function

```

The recurrence is $T(n) = T(n/2) + \Theta(1)$, which by the Master Theorem (Case 2, $a = 1$, $b = 2$, $d = 0$) gives $T(n) = \Theta(\log n)$. Problem 6 asks you to implement this and return an index or -1 .

6 Maximum-Subarray Sum via Divide and Conquer

Definition 6.1 (Maximum-subarray problem). *Given an array A of (possibly negative) numbers, find the maximum sum of any contiguous subarray $A[i : j]$.*

The divide-and-conquer solution splits A at the midpoint m . The maximum subarray either (i) lies entirely in the left half, (ii) lies entirely in the right half, or (iii) *crosses the midpoint*. Cases (i) and (ii) are solved recursively. Case (iii) is the combine step: the best crossing subarray is the best suffix of the left half joined to the best prefix of the right half, each computable by a single linear scan outward from m .

Algorithm 5 Maximum subarray (divide and conquer)

```

1: function MAXSUBARRAY( $A, lo, hi$ )
2:   if  $lo = hi$  then
3:     return  $A[lo]$ 
4:   end if
5:    $m \leftarrow \lfloor (lo + hi)/2 \rfloor$ 
6:    $left \leftarrow \text{MAXSUBARRAY}(A, lo, m)$ 
7:    $right \leftarrow \text{MAXSUBARRAY}(A, m + 1, hi)$ 
8:    $cross \leftarrow (\text{best suffix of } A[lo : m+1]) + (\text{best prefix of } A[m+1 : hi+1])$ 
9:   return  $\max(left, right, cross)$ 
10: end function

```

The combine step is $\Theta(n)$, so $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$ (Master Theorem, Case 2). (Kadane's algorithm later solves the same problem in $\Theta(n)$, but the D&C version is the cleaner illustration of the cross-boundary combine.) Problem 7 asks you to implement this and verify it against a brute-force $O(n^2)$ check.

7 Fast Exponentiation

To compute x^n for a non-negative integer n , the naive method multiplies x by itself $n - 1$ times: $\Theta(n)$ multiplications. **Exponentiation by squaring** does it in $\Theta(\log n)$ by exploiting

$$x^n = \begin{cases} 1 & n = 0, \\ (x^{n/2})^2 & n \text{ even}, \\ x \cdot (x^{(n-1)/2})^2 & n \text{ odd}. \end{cases}$$

Each recursive call halves n , so the recurrence is $T(n) = T(n/2) + \Theta(1) = \Theta(\log n)$ (Master Theorem, Case 2). Crucially, we compute $x^{n/2}$ *once* and square it – not twice – which is what makes it logarithmic rather than linear.

Problem 8 asks you to implement this with a *multiplication counter* and confirm the count is $\Theta(\log n)$ – roughly $\log_2 n$ to $2 \log_2 n$ multiplications.

Algorithm 6 Fast exponentiation (exponentiation by squaring)

```
1: function FASTPOW( $x, n$ )
2:   if  $n = 0$  then
3:     return 1
4:   end if
5:    $h \leftarrow \text{FASTPOW}(x, \lfloor n/2 \rfloor)$ 
6:   if  $n$  is even then
7:     return  $h \cdot h$ 
8:   else
9:     return  $x \cdot h \cdot h$ 
10:  end if
11: end function
```

8 Quickselect: Expected Linear-Time Selection

Definition 8.1 (Selection problem). *Given an unsorted array A and an integer k ($0 \leq k < n$), find the k -th smallest element (the element that would be at index k if A were sorted).*

Sorting and indexing gives an $O(n \log n)$ solution, but we can do better in *expectation*. **Quickselect** (Hoare) is a divide-and-conquer cousin of quicksort: pick a *pivot*, partition the array into elements less than, equal to, and greater than the pivot, and then recurse into *only the one part* that must contain the k -th smallest element.

Algorithm 7 Quickselect

```
1: function QUICKSELECT( $A, k$ )
2:   choose a pivot  $p$  from  $A$ 
3:    $less \leftarrow [a \in A : a < p];$    $equal \leftarrow [a \in A : a = p];$    $greater \leftarrow [a \in A : a > p]$ 
4:   if  $k < |less|$  then
5:     return QUICKSELECT( $less, k$ )
6:   else if  $k < |less| + |equal|$  then
7:     return  $p$ 
8:   else
9:     return QUICKSELECT( $greater, k - |less| - |equal|$ )
10:  end if
11: end function
```

Theorem 8.2. *With a uniformly random pivot, quickselect runs in $\Theta(n)$ expected time.*

Proof idea. A random pivot lands, with constant probability, in the "central" half of the values, so it discards at least a quarter of the elements with probability $\geq 1/2$. The expected work then satisfies $T(n) \leq T(3n/4) + O(n)$ in expectation – a *decreasing* geometric series $O(n)(1 + \frac{3}{4} + (\frac{3}{4})^2 + \dots) = O(n)$. (This is an unbalanced recurrence that the Master Theorem does not directly cover, because only one subproblem is solved and its size is a constant *fraction* rather than n/b ; we analyze it by the geometric-sum argument directly.) The worst case is $\Theta(n^2)$ when pivots are consistently extreme, but a random or median-of-medians pivot avoids this. $\square$

A *deterministic* pivot (e.g. always the last element) is fine for the exercises and still correct; Problem 9 asks you to implement quickselect and verify it against `sorted(A)[k]`. The same

partition-and-recurse idea, combined with a divide-and-conquer *combine*, also solves the **majority element** problem (Problem 10) and **1-D peak finding** (Problem 12), both below.

9 Two More Divide-and-Conquer Patterns

9.1 Majority element

A *majority element* of an array of length n is a value occurring more than $n/2$ times. The divide-and-conquer solution: if the left and right halves have the same majority candidate, that is the answer; otherwise, count occurrences of each half's candidate across the whole range and return whichever actually exceeds $n/2$. The combine step is a linear count, giving $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$. (The Boyer-Moore voting algorithm later does it in $\Theta(n)$, but the D&C version reinforces the combine idea.) Problem 10 asks you to implement and verify this against direct counting.

9.2 Peak finding in 1-D

An index i is a *peak* of array A if $A[i]$ is $\geq$ its neighbours (treating out-of-bounds neighbours as $-\infty$). A peak always exists, and we can find *one* in $\Theta(\log n)$: look at the middle element; if it is smaller than its right neighbour, a peak must exist to the right (the array is "rising" rightward and is bounded), so recurse right; symmetrically for the left; otherwise the middle is itself a peak. The recurrence is $T(n) = T(n/2) + \Theta(1) = \Theta(\log n)$. Problem 12 asks you to implement this.

10 Karatsuba vs. Grade-School Multiplication (a recurrence comparison)

Multiplying two n -digit integers by the grade-school method, split via divide and conquer, naively yields four half-size multiplications: $T(n) = 4T(n/2) + \Theta(n) = \Theta(n^2)$ (Master Theorem, Case 1, since $\log_2 4 = 2$). **Karatsuba's** insight is that the four products can be computed with only *three* half-size multiplications (plus some additions), giving $T(n) = 3T(n/2) + \Theta(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$ – asymptotically faster. Problem 11 asks you to *count multiplications* for both recurrences ($3T(n/2)$ vs. $4T(n/2)$) and confirm the predicted gap between $n^{\log_2 3}$ and n^2 , without implementing full big-integer arithmetic (we return to the full Karatsuba algorithm next week).

11 Summary and Looking Ahead

This week introduced the **divide-and-conquer paradigm** – divide, conquer, combine – and the machinery for analyzing it:

- **Mergesort:** sort by splitting, recursing, and merging; $\Theta(n \log n)$ via the recurrence $T(n) = 2T(n/2) + \Theta(n)$.
- **Counting inversions:** piggyback on mergesort to count cross-pairs during the merge, in $\Theta(n \log n)$.
- **Solving recurrences:** unrolling, recursion trees, and the **Master Theorem** with its three cases (leaves dominate / balanced / root dominates).

- **Binary search** ($\Theta(\log n)$), **maximum subarray** ($\Theta(n \log n)$ via a cross-boundary combine), **fast exponentiation** ($\Theta(\log n)$ multiplications), **quickselect** (expected $\Theta(n)$), **majority element**, and **1-D peak finding** ($\Theta(\log n)$).

The unifying lesson: a clever *combine* step plus recursion turns many quadratic problems into $n \log n$ (or better), and the Master Theorem tells us, at a glance, which of the three regimes a recurrence falls into.

Next week (Divide and Conquer II, Kleinberg & Tardos Ch. 5 continued) we push these ideas further: the **closest-pair-of-points** algorithm (a beautiful $\Theta(n \log n)$ geometric divide-and-conquer with a subtle combine step), the full **Karatsuba** integer-multiplication algorithm, and the **Fast Fourier Transform (FFT)** for multiplying polynomials in $\Theta(n \log n)$.